

Stabilization Analyses

Description

This file enables stabilization analyses for the feature value estimations. In other words, feature values are not estimated for a single, pre-defined number of tokens, but are estimated for discrete steps (defined with “stepsize”), and for a given maximum number of tokens (defined as “n”).

Session Info

Give the session info (reduced).

```
## [1] "R version 4.4.0 (2024-04-24)"  
## [1] "x86_64-pc-linux-gnu"
```

Load libraries

If the libraries are not installed yet, you need to install them using, for example, the command: `install.packages("ggplot2")`. For the Hrate package this is different, since it comes from github. The devtools library needs to be installed, and then the `install_github()` function is used.

```
library(stringr)  
library(ggplot2)  
library(plyr)  
library(dplyr)
```

```
##  
## Attaching package: 'dplyr'  
  
## The following objects are masked from 'package:plyr':  
##  
##   arrange, count, desc, failwith, id, mutate, rename, summarise,  
##   summarize  
  
## The following objects are masked from 'package:stats':  
##  
##   filter, lag  
  
## The following objects are masked from 'package:base':  
##  
##   intersect, setdiff, setequal, union
```

```
library(entropy)  
library(ggExtra)  
library(gsubfn)
```

```
## Loading required package: proto
```

```
# library(devtools)
# install_github("dimalik/Hrate")
library(Hrate)
```

Give the package versions.

```
##      Hrate      gsubfn      proto      ggExtra      entropy      dplyr      plyr      ggplot2
##      "0.2"      "0.7"      "1.0.0" "0.10.1" "1.3.1" "1.1.4" "1.8.9" "3.5.1"
##      stringr
##      "1.5.1"
```

External functions

```
# repetition rate function rrate()
source("~/Github/PaleoSigns/code/functions/repRate_function.R")
```

Load data

Read the preprocessed files with sequences per row.

```
# give file paths to the files to be processed
teddi <- read.csv("~/Github/PaleoSigns/data/TEDDi/final_sample.csv")
signbase <- read.csv("~/Github/PaleoSigns/data/signBase/signBase_randomized.csv")
cdli.urukV <- read.csv("~/Github/PaleoSigns/data/CDLI/UrukV/signs_clean.csv")
cdli.urukIV <- read.csv("~/Github/PaleoSigns/data/CDLI/UrukIV/signs_clean.csv")
cdli.urukIII <- read.csv("~/Github/PaleoSigns/data/CDLI/UrukIII/signs_clean.csv")
```

Preprocessing

```
# TeDDi
# select relevant columns
teddi.selected <- teddi[c("filename", "text", "writing_system")]
# rename columns
colnames(teddi.selected) <- c("id", "sequence", "subcorpus")
# add column which identifies the corpus
teddi.selected$corpus <- rep("TeDDi", nrow(teddi.selected))

# SignBase
signbase <- signbase[signbase$archaeological_unit == "Aurignacien", ]
signbase.selected <- signbase[c("object_id", "archaeological_unit", "site_name", "coding_clean")]
colnames(signbase.selected) <- c("id", "corpus", "subcorpus", "sequence")

# Proto-Cuneiform

## Uruk V
cdli.urukV.selected <- cdli.urukV[c("tablet.id", "signs.expand.vec")]
colnames(cdli.urukV.selected) <- c("id", "sequence")
cdli.urukV.selected$corpus <- rep("Proto-Cuneiform", nrow(cdli.urukV.selected))
```

```

cdli.urukV.selected$subcorpus <- rep("Uruk V", nrow(cdli.urukV.selected))

## Uruk IV
cdli.urukIV.selected <- cdli.urukIV[c("tablet.id", "signs.expand.vec")]
colnames(cdli.urukIV.selected) <- c("id", "sequence")
cdli.urukIV.selected$corpus <- rep("Proto-Cuneiform", nrow(cdli.urukIV.selected))
cdli.urukIV.selected$subcorpus <- rep("Uruk IV", nrow(cdli.urukIV.selected))

## Uruk III
cdli.urukIII.selected <- cdli.urukIII[c("tablet.id", "signs.expand.vec")]
colnames(cdli.urukIII.selected) <- c("id", "sequence")
cdli.urukIII.selected$corpus <- rep("Proto-Cuneiform", nrow(cdli.urukIII.selected))
cdli.urukIII.selected$subcorpus <- rep("Uruk III", nrow(cdli.urukIII.selected))

# Combined Data Frame
data.combined <- rbind(teddi.selected, signbase.selected, cdli.urukV.selected,
                      cdli.urukIV.selected, cdli.urukIII.selected)

```

Stabilization Analysis per Sequence

```

# set counter
counter = 0
# set the maximal number of tokens (max.n), the minimum number (min.n),
# and the stepsize for stabilization analysis
# (i.e. in steps of how many units are values calculated?).
# Note: it should be that min.n >= stepsize
max.n = 50
min.n = 2
stepsize = 2
# initialize dataframe to append results to
stabilization.df <- data.frame(id = character(0), corpus = character(0),
                              subcorpus = character(0), sequence = character(0),
                              huni.chars = numeric(0), hrate.chars = numeric(0),
                              ttr.chars = numeric(0), rm.chars = numeric(0),
                              num.tokens = numeric(0))

# start time
start_time <- Sys.time()
for (i in 1:nrow(data.combined))
{
  try({ # if the processing fails for a certain file, there will be no output for this file,
    # but the try() function allows the loop to keep running

    # get meta information per row
    row.data <- data.combined[i, ]
    id <- row.data$id
    corpus <- row.data$corpus
    subcorpus <- row.data$subcorpus
    sequence <- as.character(row.data$sequence)
    num.tokens <- nchar(as.character(row.data$sequence))

    # if/else statement to distinguish between the proto-cuneiform signs

```

```

# and individual UTF-8 characters in the other sequences
if (corpus == "Proto-Cuneiform"){
  chars <- unlist(strsplit(sequence, " "))
} else {
  chars <- unlist(strsplit(sequence, ""))
}

# preprocess text with bag-of-words normalization
# note: this is an important step! Without it, entropy rate estimation will fail
# to yield realistic results due to issues with processing special characters.
chars <- PreprocessText(chars)

# maximal length in chars
chars <- chars[1:max.n] # use only maximally n units
# remove NAs for vectors which are already shorter than max.n
chars <- chars[!is.na(chars)]

# use "next" statement to exclude files with less than x characters
if (length(chars) < min.n) {
  next
}

# run loop with stepsizes
# define the number of units (i.e. characters) used for analyses (note that k is
# always either equal to or smaller than n)
k = length(chars)
for (j in 1:(k/stepsize))
{
  # unigram entropy estimation
  # define substring of chars vector by stepsize
  subchars <- chars[1:(j*stepsize)]
  # calculate unigram entropy for characters
  subchars.df <- as.data.frame(table(subchars))
  # print(chars.df)
  huni.chars <- entropy(subchars.df$Freq, method = "ML", unit = "log2")
  # entropy rate estimation
  # note: the values chosen for max.length and every.word will crucially
  # impact processing time. max.length = NULL means all units in the file are
  # considered.
  hrates.chars <- get.estimate(text = subchars, every.word = 1,
                              max.length = NULL)
  # calculate type-token ratio (ttr)
  ttr.chars <- nrow(subchars.df)/sum(subchars.df$Freq)

  # calculate repetition rate
  rm.chars <- rrate(subchars, method = "lsmo")

  # append results to dataframe
  local.df <- data.frame(id, corpus, subcorpus, sequence,
                        huni.chars, hrates.chars,
                        ttr.chars, rm.chars, num.tokens = j*stepsize)
  stabilization.df <- rbind(stabilization.df, local.df)
}

```

```

# counter
counter <- counter + 1
#print(counter)
})
}
end_time <- Sys.time()
end_time - start_time

```

```
## Time difference of 3.151914 mins
```

```
head(stabilization.df)
```

```

##           id corpus subcorpus
## 1 amp_nfi_1.txt  TeDDi      Latn
## 2 amp_nfi_1.txt  TeDDi      Latn
## 3 amp_nfi_1.txt  TeDDi      Latn
## 4 amp_nfi_1.txt  TeDDi      Latn
## 5 amp_nfi_1.txt  TeDDi      Latn
## 6 amp_nfi_1.txt  TeDDi      Latn
##
## 1 Na nd mounmotrhu mrokfot inji wanyhato duka memoya nhai wom hefenr tohyuk frohrn o bbinysotf tohyul
## 2 Na nd mounmotrhu mrokfot inji wanyhato duka memoya nhai wom hefenr tohyuk frohrn o bbinysotf tohyul
## 3 Na nd mounmotrhu mrokfot inji wanyhato duka memoya nhai wom hefenr tohyuk frohrn o bbinysotf tohyul
## 4 Na nd mounmotrhu mrokfot inji wanyhato duka memoya nhai wom hefenr tohyuk frohrn o bbinysotf tohyul
## 5 Na nd mounmotrhu mrokfot inji wanyhato duka memoya nhai wom hefenr tohyuk frohrn o bbinysotf tohyul
## 6 Na nd mounmotrhu mrokfot inji wanyhato duka memoya nhai wom hefenr tohyuk frohrn o bbinysotf tohyul
##   huni.chars hrates.chars ttr.chars rm.chars num.tokens
## 1   1.000000    0.500000 1.0000000      0          2
## 2   2.000000    1.146241 1.0000000      0          4
## 3   2.251629    1.366562 0.8333333      0          6
## 4   2.750000    1.750841 0.8750000      0          8
## 5   2.921928    1.883762 0.8000000      0         10
## 6   2.918296    1.815270 0.6666667      0         12

```

Stabilization plots

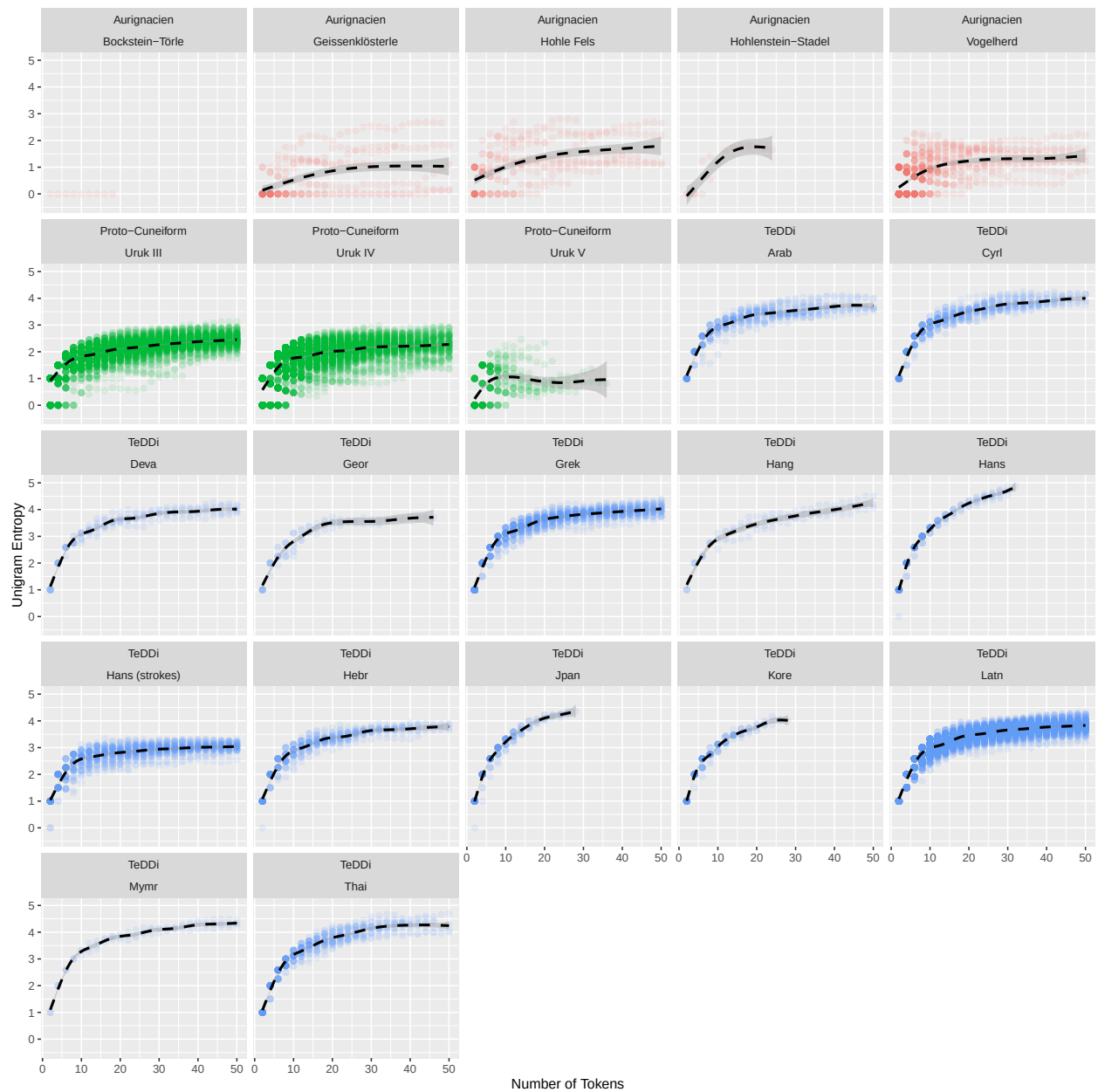
Unigram entropy

```

huni.chars.plot <- ggplot(stabilization.df, aes(x = num.tokens, y = huni.chars,
                                                color = corpus)) +
  geom_point(alpha = 0.1, size = 2) +
  geom_smooth(color = "black", linetype = 2) +
  theme(legend.position = "none") +
  labs(x = "Number of Tokens", y = "Unigram Entropy") +
  facet_wrap(~ corpus + subcorpus)
huni.chars.plot

```

```
## `geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```



Safe figure to file

```
ggsave("~/Github/PaleoSigns/figures/stabilization_huni_chars.pdf", huni.chars.plot, dpi = 300,
        width = 12, height = 12,
        scale = 1, device = cairo_pdf)
```

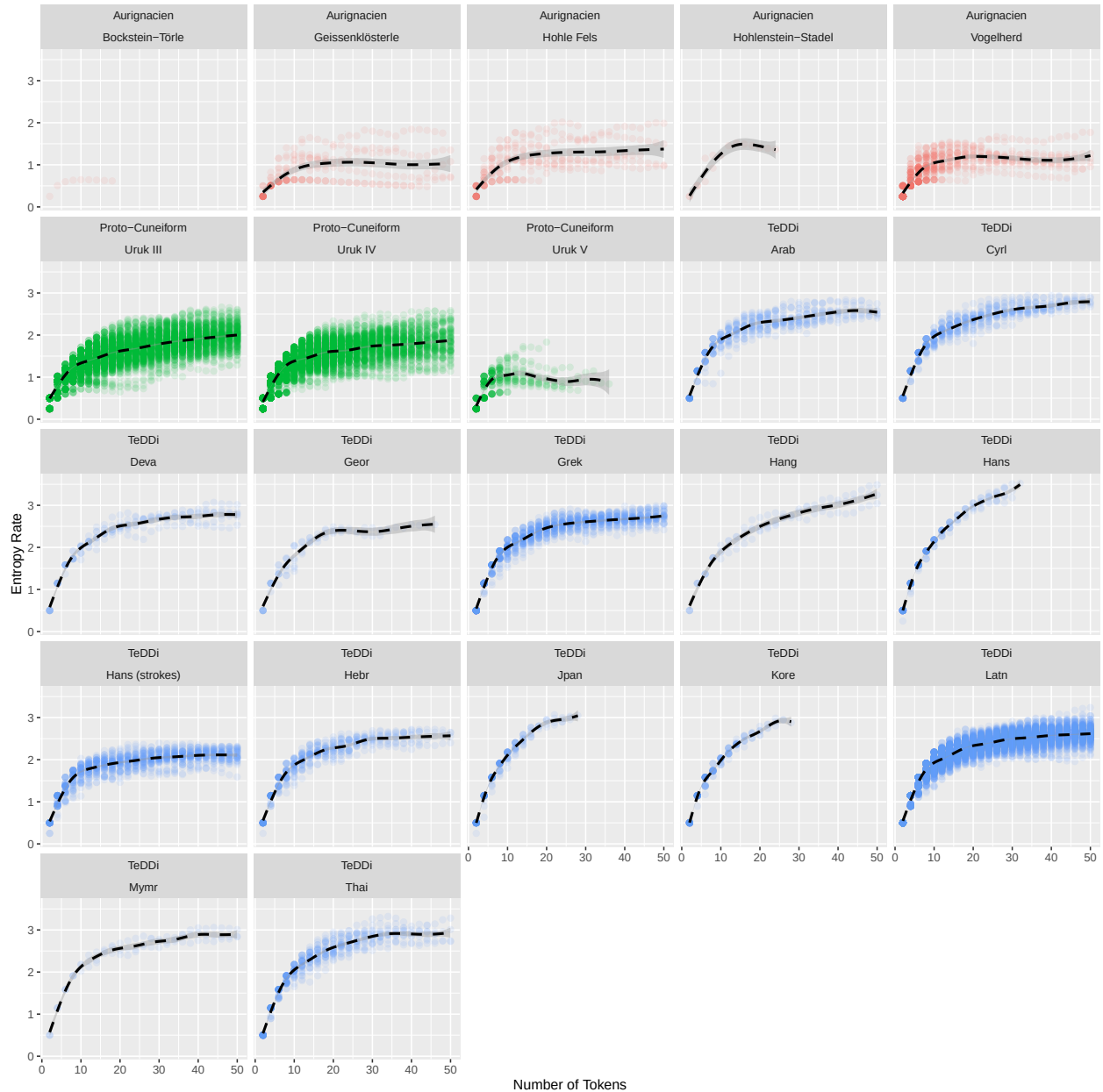
```
## `geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```

Entropy rate

```
hrate.chars.plot <- ggplot(stabilization.df, aes(x = num.tokens, y = hrate.chars,
                                                colour = corpus)) +

  geom_point(alpha = 0.1, size = 2) +
  geom_smooth(color = "black", linetype = 2) +
  theme(legend.position = "none") +
  labs(x = "Number of Tokens", y = "Entropy Rate") +
  facet_wrap(~ corpus + subcorpus)
hrate.chars.plot
```

```
## `geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```



Safe figure to file

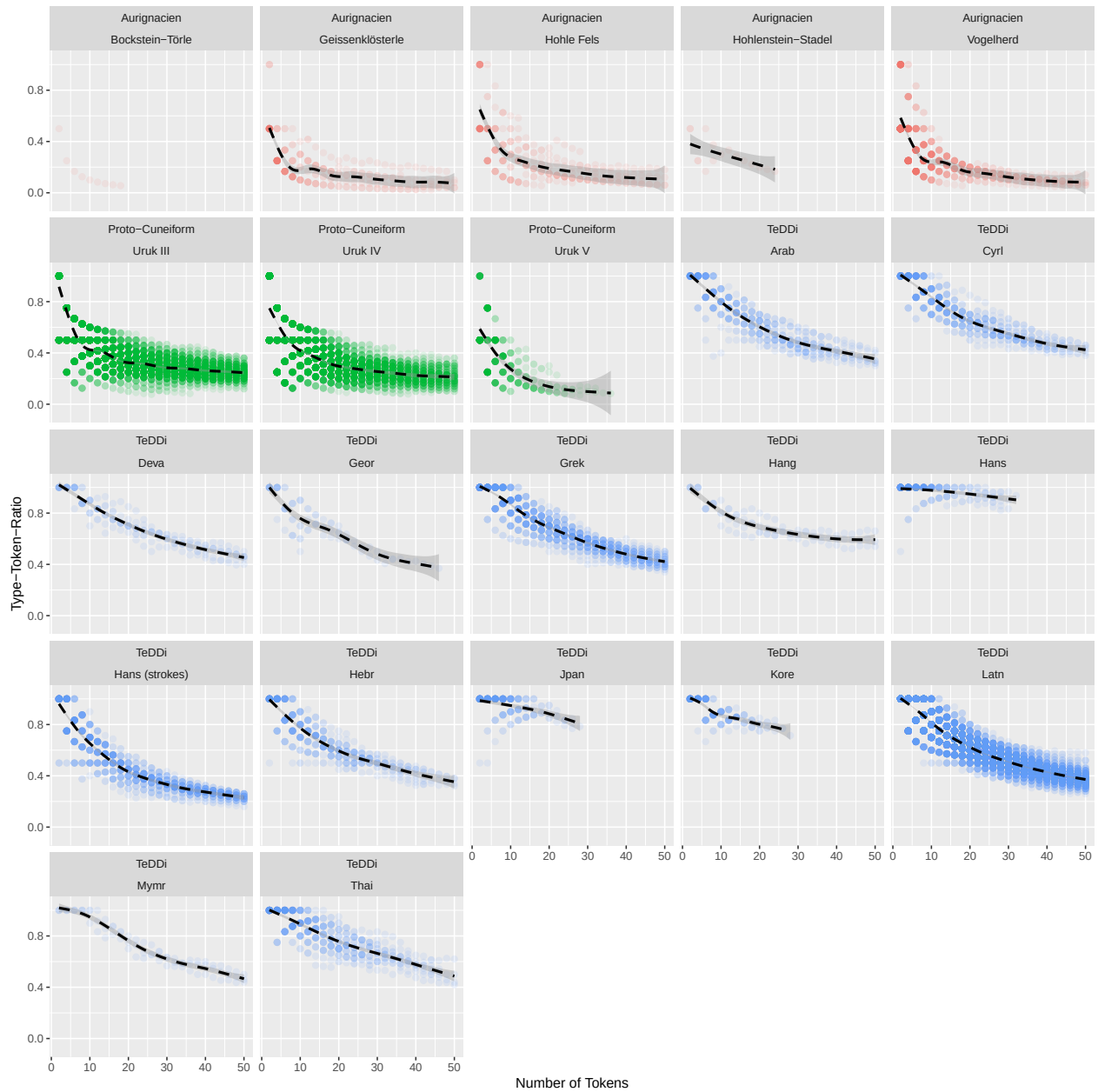
```
ggsave("~/Github/PaleoSigns/figures/stabilization_hrate_chars.pdf", hrate.chars.plot, dpi = 300,  
        width = 12, height = 12,  
        scale = 1, device = cairo_pdf, limitsize = F)
```

```
## `geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```

TTR

```
ttr.chars.plot <- ggplot(stabilization.df, aes(x = num.tokens, y = ttr.chars,  
                                              colour = corpus)) +  
  geom_point(alpha = 0.1, size = 2) +  
  geom_smooth(color = "black", linetype = 2) +  
  theme(legend.position = "none") +  
  labs(x = "Number of Tokens", y = "Type-Token-Ratio") +  
  facet_wrap(~ corpus + subcorpus)  
ttr.chars.plot
```

```
## `geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```

Safe figure to file

```
ggsave("~/Github/PaleoSigns/figures/stabilization_ttr_chars.pdf", ttr.chars.plot, dpi = 300,
        width = 12, height = 12,
        scale = 1, device = cairo_pdf)
```

```
## `geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```

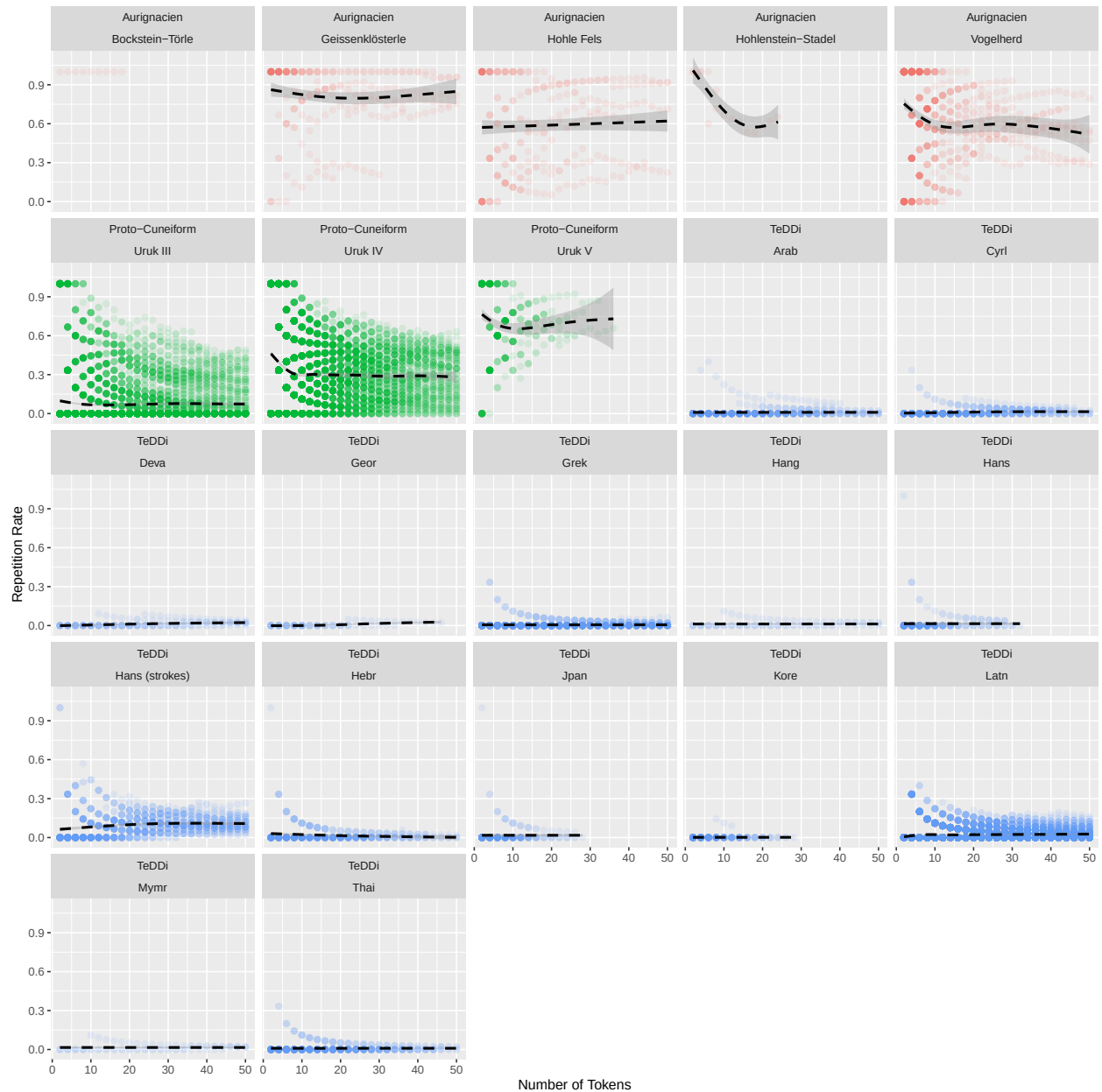
Repetition rate

```
rm.chars.plot <- ggplot(stabilization.df, aes(x = num.tokens, y = rm.chars,
                                              colour = corpus)) +

  geom_point(alpha = 0.1, size = 2) +
  geom_smooth(color = "black", linetype = 2) +
  theme(legend.position = "none") +
  labs(x = "Number of Tokens", y = "Repetition Rate") +
  facet_wrap(~ corpus + subcorpus)

rm.chars.plot
```

```
## `geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```



Safe figure to file

```
ggsave("~/Github/PaleoSigns/figures/stabilization_rm_chars.pdf", rm.chars.plot, dpi = 300,  
        width = 12, height = 12,  
        scale = 1, device = cairo_pdf)
```

```
## `geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```

Package References

```
## Bentz C, Alikaniotis D (2017). _Hrate: Implements LZ estimator for  
## texts_. R package version 0.2, commit  
## b67b4bb5f5430ce42d2844da30ddf5642540aa30,  
## <https://github.com/dimalik/Hrate>.  
  
## Wickham H (2023). _stringr: Simple, Consistent Wrappers for Common  
## String Operations_. R package version 1.5.1,  
## <https://CRAN.R-project.org/package=stringr>.  
  
## Grothendieck G (2018). _gsubfn: Utilities for Strings and Function  
## Arguments_. R package version 0.7,  
## <https://CRAN.R-project.org/package=gsubfn>.  
  
## Hausser J, Strimmer K (2021). _entropy: Estimation of Entropy, Mutual  
## Information and Related Quantities_. R package version 1.3.1,  
## <https://CRAN.R-project.org/package=entropy>.  
  
## Wickham H (2016). _ggplot2: Elegant Graphics for Data Analysis_.  
## Springer-Verlag New York. ISBN 978-3-319-24277-4,  
## <https://ggplot2.tidyverse.org>.  
  
## Wickham H (2011). "The Split-Apply-Combine Strategy for Data Analysis."  
## _Journal of Statistical Software_, *40*(1), 1-29.  
## <https://www.jstatsoft.org/v40/i01/>.  
  
## Attali D, Baker C (2023). _ggExtra: Add Marginal Histograms to  
## 'ggplot2', and More 'ggplot2' Enhancements_. R package version 0.10.1,  
## <https://CRAN.R-project.org/package=ggExtra>.
```